

100 Days of Azure DevOps

Day 29 Handout — `$( )` is runtime. `${{ }}` is compile-time. A secret in YAML is a postcard

3 - Continuous Integration · Personal lab only · Educational content · Not a sales pitch

Architecture A — where a value lives and when it expands

Store	Expansion	Safe for secrets?
YAML variables: key: value	<code>\$(key)</code> at runtime; <code>\${{ variables.key }}</code> at compile	No. Repo is readable
UI pipeline variable	<code>\$(name)</code> runtime; secret ones mask as <code>***</code>	Only if marked secret; still not Key Vault
Library variable group	variables: - group: lab-common	Non-secrets yes; secrets if issecret + mask
Group linked to Key Vault	Preview today; deep dive Day 64–65	Yes — vault is source, group is pointer
Compile vs runtime	<code>\${{ }}</code> baked when the run is queued	Do not <code>\${{ }}</code> a secret into a condition string

Architecture B — pick the drawer, not a blog title

Need	Use	Skip
App name, region, SKU label	Group lab-common → <code>\$(appName)</code>	Hardcoded per-pipeline copies
Dummy lab secret	Group secret labDummy value: not-a-password	Real password, PAT, or connection string
Same value on 4 pipelines	One group, many YAML references	Four UI variables you will drift
Production secret later	Key Vault + RBAC (Phase 7)	Putting it in azure-pipelines.yml 'just for now'

Architecture C — pitfalls that print the postcard

Pitfall	What you see	Fix
<code>echo \$(labDummy)</code>	Clear text if not marked secret	Mark secret; debug length only, never letters
Group not linked	<code>\$(appName)</code> empty or literal <code>\$(appName)</code>	variables: - group: lab-common on the pipeline
Auth on the group	Job fails: variable group could not be accessed	Pipeline must be authorized to the Library group
Secret in compile-time	Value appears in expanded YAML of the run	Keep secrets runtime-only; never <code>\${{ }}</code> them

One-liner to remember

Secrets in YAML are postcards — variable groups and Key Vault are envelopes. If you need to debug, print the length, not the letters.

Step-by-step lab (personal Azure / Azure DevOps only)

Personal Azure DevOps org only. Project azure-100-labs. Dummy values only — never a real password, PAT, or work secret.

- 1 Pipelines → Library → Variable groups → New → name lab-common. Add `appName=azure-100-labs` (not secret).
- 2 Add labDummy as a secret. Value: not-a-real-password (literally). Do not reuse anything from work.
- 3 In YAML set variables: - group: lab-common. First run will ask to authorize the group — allow it for this project only.
- 4 Step: echo App name is `$(appName)`. Confirm it expands. Do not echo `$(labDummy)`.
- 5 Optional: echo Dummy length is `${#LAB}` after mapping — or skip and just confirm the secret shows as `***` if you print it by accident, then remove that line.
- 6 Write docs/variables-day29.md: compile-time `${{ }}` vs runtime `$( )` in two sentences. No values in the doc.

Starter YAML — group reference, no secret echo

```
# pipelines/vars-day29.yml
trigger: none
variables:
- group: lab-common
pool:
  vmImage: ubuntu-latest
steps:
- script: echo "App name is $(appName)"
  displayName: Non-secret from group
- script: echo "Dummy is set but will not be printed"
  displayName: Secret stays in the envelope
```

Done checklist

- Group lab-common exists; pipeline authorized to it
- \$(appName) expanded in the log
- No real secret, PAT, or connection string in YAML or screenshots
- Can explain \${ } vs \$() without saying 'variables are variables'

Tomorrow — Day 30

Phase 3 mini project — one green CI on your primary stack, artifact in hand, secrets still out of the file.